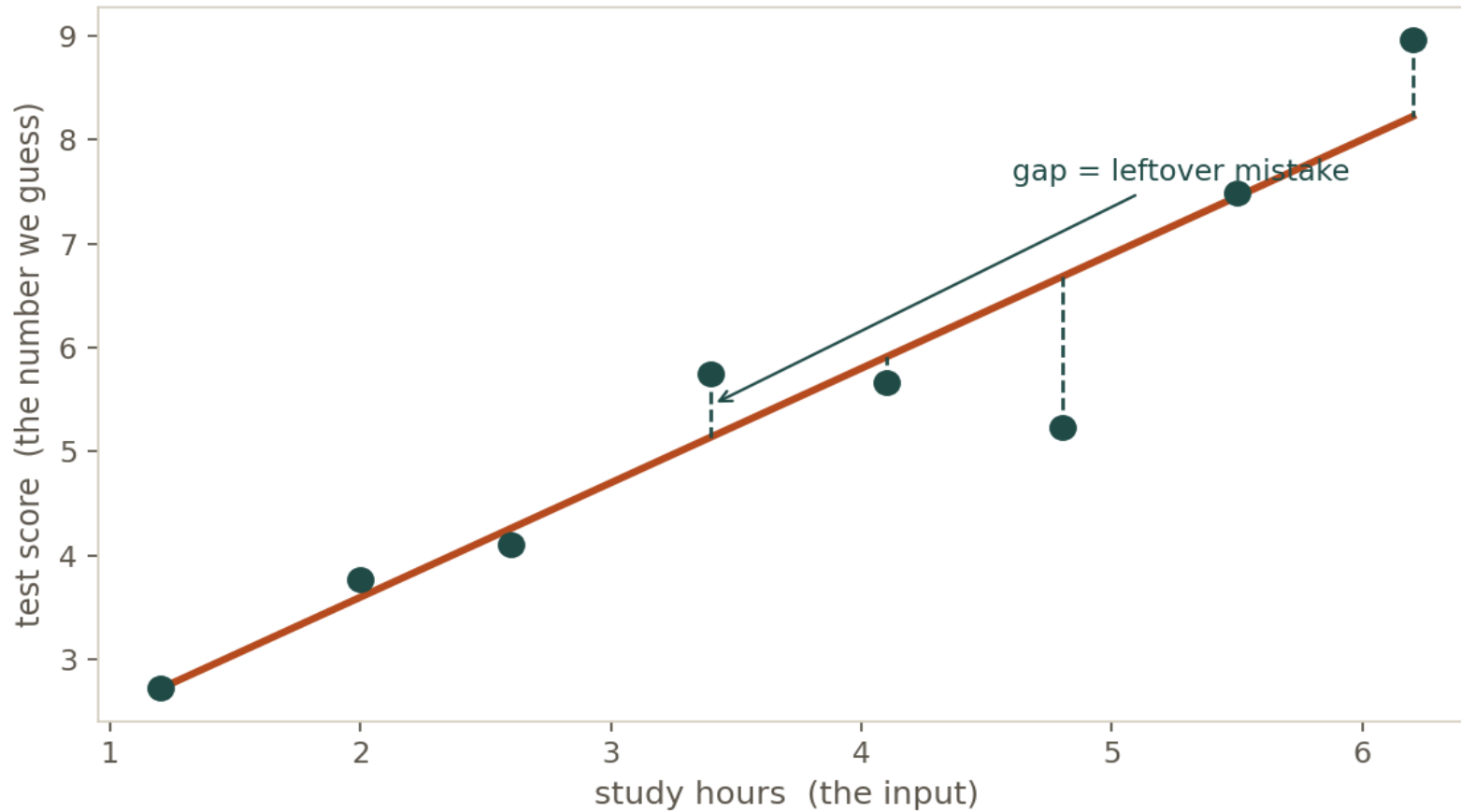


IN404 UNIT 2

LINEAR REGRESSION

Linear regression = the straight line that fits the dots



Simple (one input)

$$\hat{y} = b_0 + b_1x$$

In words: guess = starting height + slope $\times$ input

- $\hat{y}$ = the number we guess
- x = the input (homework, hours, square feet)
- b_1 = the slope (how much y moves when x goes up by 1)
- b_0 = the intercept (the guess when x is 0)

How the computer picks b_0 and b_1

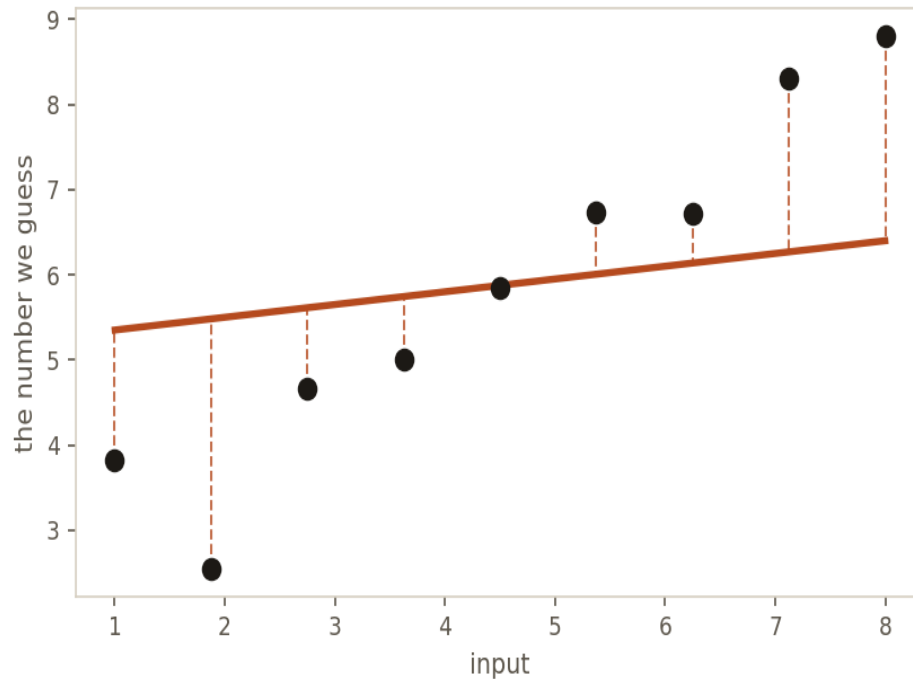
It shrinks the leftover mistakes (least squares):

$$\text{error} = y - \hat{y}$$

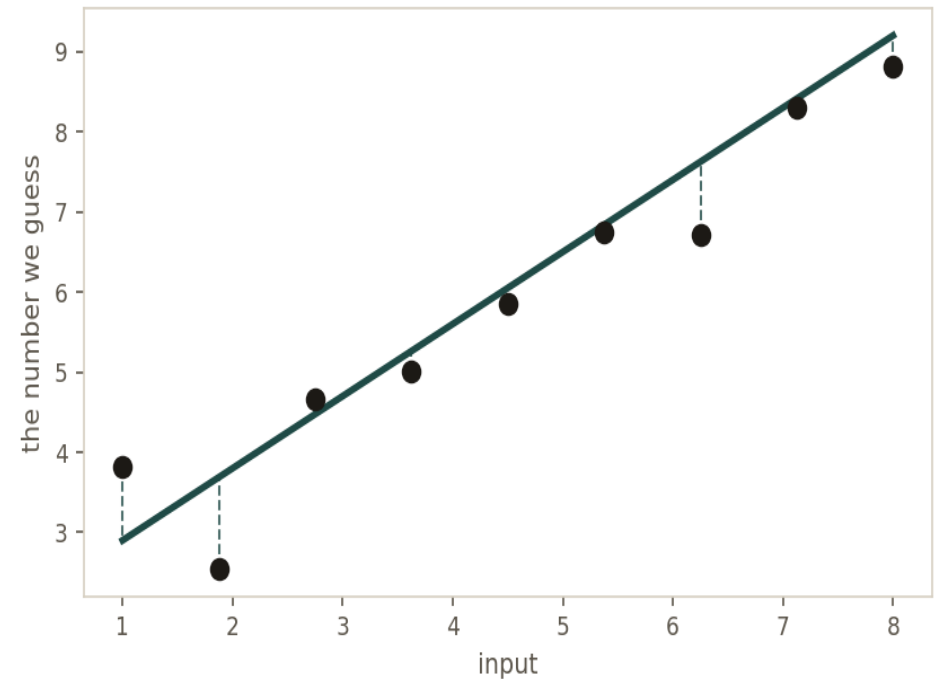
find b_0, b_1 that make $\sum (y - \hat{y})^2$ as small as possible

The computer tries lines until the leftover mistakes are smallest

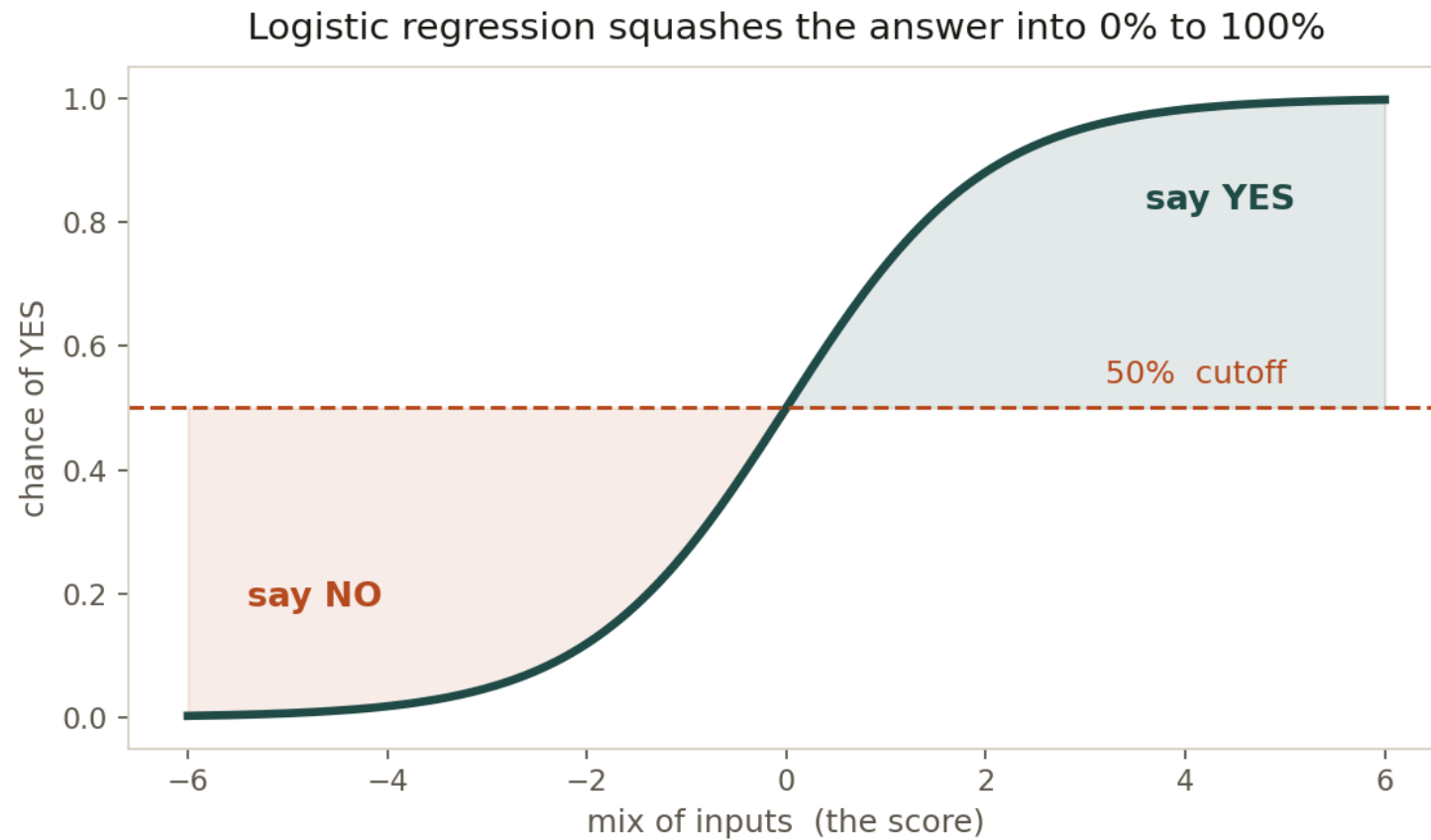
A bad line (big leftover gaps)



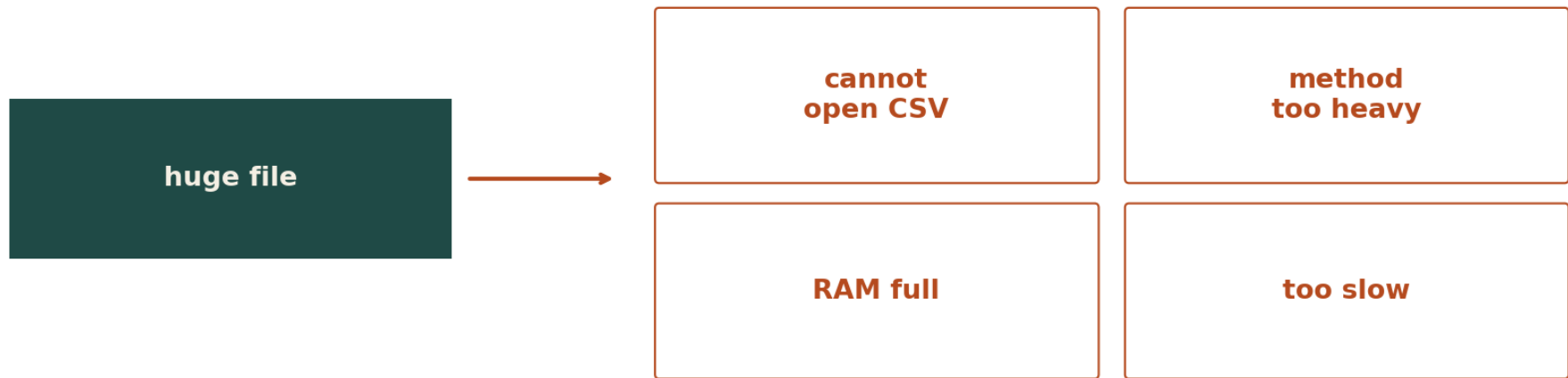
The best line (gaps as small as they can get)



Now the answer is YES or NO



If the file grows 100 times, does it still finish?



Four jams: RAM fills up, a Python loop is too slow, you cannot open the whole CSV, or the method itself is too heavy. Scale means it still finishes and still fits in memory.

Three boxes. One inside the next.

ARTIFICIAL INTELLIGENCE - the big box

Any time a machine does something that looks smart: following rules, searching a map, or learning from examples.

MACHINE LEARNING - it learned from examples

Show it houses and prices, it finds the line. Show it blood tests and malaria, it finds the S-curve. Linear and logistic live here.

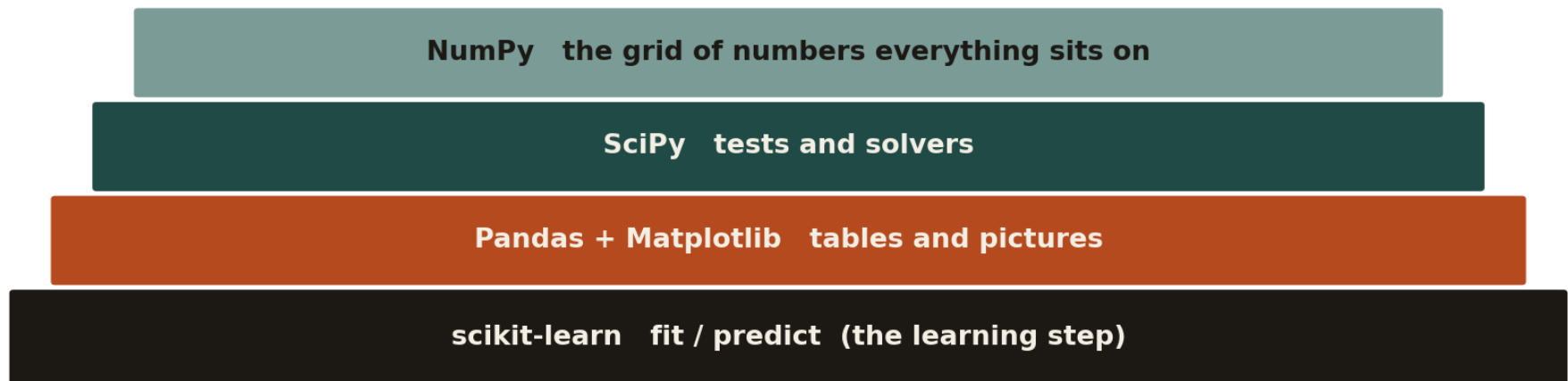
DEEP LEARNING - stacked layers

Many layers of artificial neurons. Good at pictures, speech, and language. Wants a lot of data and a lot of computer power.

ITEM 6 | PICTURE

The toolbox is a stack. Bottom supports the top.

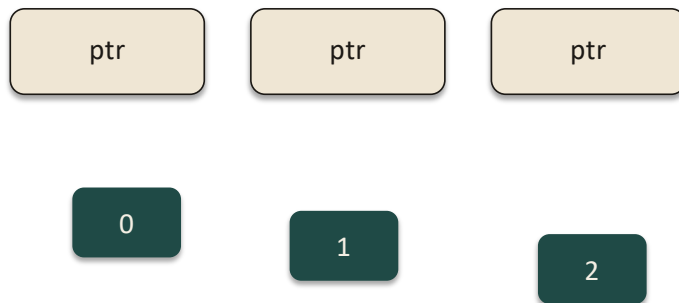
Python scientific stack (bottom supports the top)



NumPy is the grid. SciPy does extra math. Pandas and Matplotlib handle tables and pictures. scikit-learn does the learning.

Same numbers. Different memory.

Python list = a row of arrows



Each arrow points to a separate Python object. Objects can be mixed: a number, a word, a 3.2. Easy to grow. Bad at math. `my_list + 1` crashes.

NumPy array = one packed row



This is d and e in IN404_Unit2.py (0 through 7)

One block of the SAME kind of number, packed side by side. `e+1` works. `e**2` works. Faster, less memory, and every other library in this unit speaks this format.